

Wydział: W4N
Rok: 2025/2026
Semestr: 5

Data oddania pracy:
22 stycznia 2026

SYSTEMY OPERACYJNE

Projekt wielowątkowego serwera usług z obsługą
priorytetów i kontrolą dostępu do zasobów

Autor:
Kacper Piekut 280879

Prowadzący:
dr Marek Bazan



Politechnika Wrocławska

1. Wstęp

Współbieżność w systemach operacyjnych i aplikacjach serwerowych jest kluczowym zagadnieniem pozwalającym zwiększać przepustowość oraz skracać czasy odpowiedzi poprzez równoległe wykonywanie zadań. Jednocześnie wprowadza ona szereg wyzwań teoretycznych i praktycznych, takich jak warunki wyścigu (Race Conditions), zakleszczenia (Deadlocks), zagłodzenie (Starvation) czy zapewnienie spójności danych przy jednoczesnym dostępie wielu wątków. Rozwiązanie tych problemów wymaga stosowania odpowiednich prymitywów synchronizacji oraz świadomego projektowania architektury. Do najważniejszych mechanizmów synchronizacji należą blokady wzajemne (`std::mutex`) zapewniające wyłączny dostęp do sekcji krytycznych, blokady współdzielone (`std::shared_mutex`) pozwalające wielu czytelnikom jednocześnie odczytywać zasób przy zachowaniu wyłączności dla pisarza, warunki (`std::condition_variable`) umożliwiające efektywne oczekiwanie na zdarzenia. W praktyce serwerowej często stosuje się również pule wątków (Thread Pool), które pozwalają kontrolować liczbę aktywnych wątków i równomiernie rozdzielać pracę, oraz kolejki priorytetowe do porządkowania zadań według ważności.

2. Cele projektu

Celem projektu jest stworzenie i analiza wielowątkowego serwera usług w języku C++, który demonstruje poprawne i efektywne mechanizmy synchronizacji na dwóch poziomach - globalnym i lokalnym - oraz wpływ priorytetów zadań na działanie systemu. W szczególności:

- Zaprojektowanie architektury z kilkoma usługami współdzielącymi zasoby globalne.
- Implementacja lokalnego wzorca producent-konsument: każda usługa posiada własną kolejkę priorytetową zadań oraz pulę wątków roboczych.
- Implementacja ochrony zasobów globalnych zgodnie z modelem czytelnicy-pisarze - równoległe odczyty, wyłączny zapis.
- Zaimplementowanie mechanizmu łagodnego zamykania (graceful shutdown): po sygnale zatrzymania przestajemy przyjmować nowe zadania i kończymy dopiero po opróżnieniu kolejek.
- Weryfikacja spójności przez liczniki zadań (produced i done) oraz raport końcowy potwierdzający, że wszystkie zlecone zadania zostały przetworzone.

3. Plan eksperymentu

Testowany system

Przedmiotem badań jest wielowątkowy serwer usług napisany w języku C++. System składa się z wielu usług, z których każda posiada własną lokalną kolejkę priorytetową zadań oraz pulę wątków roboczych. Zasoby globalne są współdzielone i chronione odpowiednio za pomocą `std::shared_mutex` (model czytelnicy-pisarz) oraz `std::mutex`.

Środowisko testowe

Eksperyment jest wykonywany w środowisku konsolowym na systemie Windows. Kod kompilowany jest przy użyciu kompilatora MinGW (GCC) w wersji 13.2.0; uruchamianie odbywa się z poziomu konsoli systemu.

Specyfikacja sprzętowa

Laptop: ASUS TUF Gaming F15.

Procesor: Intel Core i5-11400H, 6 rdzeni, 12 wątków, taktowanie bazowe 2.7 GHz (do 4.5 GHz w trybie turbo).

Pamięć RAM: 16 GB DDR4.

Dysk: SSD NVMe 512 GB (Samsung MZVLQ512HBLU-00B00).

Karta graficzna: NVIDIA GeForce RTX 3050 Laptop GPU
oraz Intel UHD Graphics.

System operacyjny: Microsoft Windows 11 Home 64-bit, wersja 10.0.26100.

4. Implementacja

Poniżej przedstawiono szczegółowy opis implementacji programu, plik po pliku, wraz z omówieniem kluczowych klas, funkcji i zastosowanych mechanizmów synchronizacji.

4.1. Struktura projektu

Projekt znajduje się w folderze głównym i składa się z następujących plików:

- `main.cpp` - punkt wejścia programu oraz miejsce konfiguracji przebiegu. Zawiera uruchomienie domyślne programu oraz tryb testowy z trzema scenariuszami, które modyfikują tylko parametry i logi, bez zmiany architektury usług.
- `Task.hpp` - definicja zadania (`Task`) oraz komparatora priorytetów (`TaskCompare`). Ten plik określa, jakie informacje niesie zadanie (id, priorytet, dane tekstowe) i w jaki sposób kolejka rozstrzyga kolejność obsługi.
- `SafePriorityQueue.hpp` - bezpieczna wątkowo kolejka priorytetowa oparta o `std::priority_queue`. Odpowiada za synchronizację dostępu (`mutex`, `condition_variable`) oraz mechanizm zamykania (`close`), który pozwala wątkom roboczym zakończyć pracę po opróżnieniu kolejki.
- `SharedResources.hpp` - wspólne zasoby używane przez wszystkie usługi. Zawiera `Logger` (z możliwością wyciszenia logów w testach) oraz `DataPool` chroniony blokadą współdzieloną w modelu czytelnicy-pisarz.
- `Service.hpp` / `Service.cpp` - implementacja pojedynczej usługi działającej jako producent-konsument. Klasa zarządza pulą wątków roboczych, lokalną kolejką zadań, licznikami `produced/done` oraz poprawnym startem i zatrzymaniem.

4.2. Plik main.cpp

Punkt wejścia programu. Plik zawiera tryb uruchomienia domyślnego oraz tryb testowy (scenariusze), które wykorzystują ten sam przebieg programu, a różnią się jedynie parametrami i dodatkowymi logami:

- Zawiera flagę `testMode`, która decyduje czy program uruchamia się w trybie domyślnym, czy w trybie scenariuszy.
- Zawiera flagę `terminalLogMode`, która steruje wypisywaniem logów w terminalu (nie wpływa na zapisywanie logów do pliku).
- Na początku przygotowywana jest ścieżka do pliku logów: sprawdzany jest katalog `logs` (w razie potrzeby tworzony), a nazwa pliku ma format `logs-dd-mm-rrrr_gg-mm.txt` (czas uruchomienia programu z dokładnością do minuty).
- Definiuje funkcję uruchamiającą przebieg programu (`run_default_program`), która tworzy `SharedResources` oraz dwie usługi: `DataService` (50 wątków) i `ReportService` (50 wątków), a następnie uruchamia je metodą `start()`.
- W pętli generuje określoną liczbę par zadań i przekazuje je do odpowiednich usług; stosowane jest krótkie opóźnienie `sleep_for(5 ms)`, aby zasymulować napływ pracy.
- Po zakończeniu generowania zadań program odczekuje określony czas (`stopAfter`), a następnie wywołuje `stop()` na obu usługach.
- Na koniec sumuje liczniki `produced_count()` oraz `done_count()` obu usług i wypisuje raport w konsoli w formie: Wygenerowane: X Wykonane: Y Status: OK/MISMATCH.
- Po zatrzymaniu usług bufor logów jest zapisywany do pliku w katalogu `logs` poprzez wywołanie metody `dumpToFile` na obiekcie `logger`.
- Tryb testowy: program pyta o numer scenariusza (1/2/3), a następnie ustawia parametry przebiegu.
- Scenariusz 1: wielokrotne uruchomienie tego samego przebiegu (np. 10 razy) z wyciszonymi logami (Logger wyłączony), aby sprawdzić spójność (oczekujemy $X = Y$).
- Scenariusz 2: w trakcie generowania zadań do kolejki `DataService` wstrzykiwane jest dodatkowe zadanie o bardzo wysokim priorytecie wraz z logiem informacyjnym.
- Scenariusz 3: program wypisuje log informujący o zakończeniu przyjmowania nowych zadań możliwie szybko po dodaniu ostatnich zadań do kolejek, a następnie pokazuje, że wątki kończą pracę dopiero po opróżnieniu kolejki.

4.3. Plik Task.hpp

Plik definiuje strukturę zadania (Task) oraz komparator używany przez kolejkę priorytetową:

- struct Task - zawiera pola: id (liczba całkowita identyfikująca zadanie), priority (liczba całkowita określająca priorytet) oraz data (tekst).
- struct TaskCompare - implementuje operator porównania tak, aby element o większym priorytecie trafiał na szczyt std::priority_queue.

Założenia: zmienna priority jest nieujemna, lecz kod nie wymusza tego wprost; nie ma dodatkowych ograniczeń.

4.4. Plik SafePriorityQueue.hpp

Jest to szablon klasy SafePriorityQueue<T, Compare> opakowujący std::priority_queue w bezpieczny wątkowo interfejs. Wewnętrznie wykorzystywane są: std::mutex do ochrony sekcji krytycznych, std::condition_variable do efektywnego czekania na pojawienie się elementów oraz flaga closed do obsługi łagodnego zamykania. Poniżej znajdują się opisy kluczowych metod:

- void push(const T& item) - dodaje nowe zadanie do kolejki; jeśli kolejka została wcześniej zamknięta, dodawanie jest pomijane. Po wstawieniu elementu informuje czekające wątki, że pojawiły się dane do przetworzenia.
- bool try_pop(T& out) - szybka próba pobrania najważniejszego (najwyższego priorytetu) zadania bez czekania. Gdy kolejka jest pusta, natychmiast zwraca informację, że nic nie było do pobrania; w przeciwnym razie oddaje zadanie.
- bool wait_and_pop(T& out) - czeka, aż w kolejce pojawi się zadanie lub aż kolejka zostanie zamknięta. Jeśli po przebudzeniu coś jest do przetworzenia, zwraca to zadanie; gdy kolejka jest już pusta i zamknięta, sygnalizuje koniec pracy.
- void close() - zamyka kolejkę: od tej chwili nie przyjmuje nowych zadań i budzi wszystkie wątki, które mogły na nią czekać, aby mogły zakończyć działanie.
- bool empty() const, size_t size() const, bool isClosed() const - krótkie pytania o stan kolejki: czy jest pusta, ile ma elementów i czy została zamknięta.

Semantyka zamykania: wywołanie close() nie usuwa elementów z kolejki, ale pozwala wait_and_pop przerwać czekanie, gdy kolejka stanie się pusta po przetworzeniu wszystkich zadań.

4.5. Plik `SharedResources.hpp`

Plik definiuje zasoby współdzielone przez wszystkie usługi:

- **Logger** - rejestrator komunikatów oparty o buforowanie w pamięci. Dostęp jest chroniony wspólnym zamkiem (`logMtx`), aby wpisy z różnych wątków nie mieszały się ze sobą. Metoda `log(const std::string& msg)` dopisuje pojedynczą linię do bufora (vector napisów), a wypis do terminala jest kontrolowany flagą `consoleEnabled` ustawianą metodą `setConsoleEnabled(bool)`. Udostępniana jest metoda `dumpToFile(const std::filesystem::path& filePath)`, która zapisuje zawartość bufora do pliku tekstowego (jedna linia w buforze odpowiada jednej linii w pliku) oraz czyści bufor.
- **DataPool** - współdzielona tablica liczb całkowitych (`dataVector`) zabezpieczona zamkiem współdzielonym (`sharedMtx`).
 - `read(idx)` - bezpiecznie odczytuje wartość spod podanego indeksu; gdy indeks jest poza zakresem, zwraca `-1`. Wiele wątków może jednocześnie czytać.
 - `write(idx, value)` - zapisuje wartość pod wskazanym indeksem; jeśli trzeba, najpierw powiększa tablicę. Podczas zapisu ma wyłączny dostęp.
 - `size()` - zwraca bieżącą liczbę elementów w tablicy.

Model czytelnicy-pisarz: wiele odczytów może zachodzić równolegle, natomiast zapis odbywa się w odosobnieniu.

4.6. Plik `Service.hpp`

Deklaracja klasy `Service`, która reprezentuje pojedynczą usługę z własną kolejką zadań i pulą wątków. Najważniejsze pola:

- `serviceName` - etykieta (nazwa) usługi używana m.in. w logach.
- `sharedResources` - dostęp do wspólnych zasobów (`logger` i `DataPool`).
- `taskQueue` - lokalna kolejka priorytetowa zadań obsługiwanych przez usługę.
- `workers` - kontener uruchomionych wątków roboczych.
- `mtx` - zamek chroniący pola sterujące cyklem życia i liczniki.
- `producedCount`, `doneCount` - ile zadań przyjęto i ile zakończono.
- `running` - czy usługa aktualnie przyjmuje nowe zadania.

Udostępniany interfejs obejmuje m.in. `start()`, `stop()`, `submit(const Task&)`, `produced_count()`, `done_count()`, a także pomocniczą metodę `wait_until_done()` wykorzystywaną w testach.

4.7. Plik Service.cpp

Implementacja logiki usługi i cyklu życia wątków:

- `Service(std::string name, SharedResources& shared, int workerCount)` - przygotowuje miejsce na `workerCount` wątków w kontenerze `workers`; na tym etapie wątki jeszcze nie pracują.
- `Service()` - na wyjściu wywołuje `stop()`, aby domknąć usługę bez pozostawiania wątków w tle.
- `start()` - ustawia `running=true` (pod ochroną `mtx`) i uruchamia wątki z funkcją `worker_loop(int)`.
- `stop()` - przestawia `running=false`, zamyka lokalną kolejkę (`taskQueue.close()`) i dołącza wszystkie wątki, czekając aż dokończą bieżące zadania.
- `submit(const Task&)` - jeśli `running` jest aktywne, zwiększa `producedCount` i dopisuje zadanie do `taskQueue`.
- `produced_count()`, `done_count()` - zwracają liczniki z zachowaniem ochrony (`mtx`).
- `worker_loop(int)` - pętla pracy wątku: pobiera kolejne zadania z kolejki, loguje ich obsługę, losowo wykonuje odczyt lub zapis w `DataPool`, wprowadza krótką przerwę (ok. 20 ms), a następnie zwiększa `doneCount`. Gdy kolejka zostanie zamknięta i opróżniona, wątek kończy pracę i zapisuje stosowną informację w logu.

4.8. Uwagi końcowe

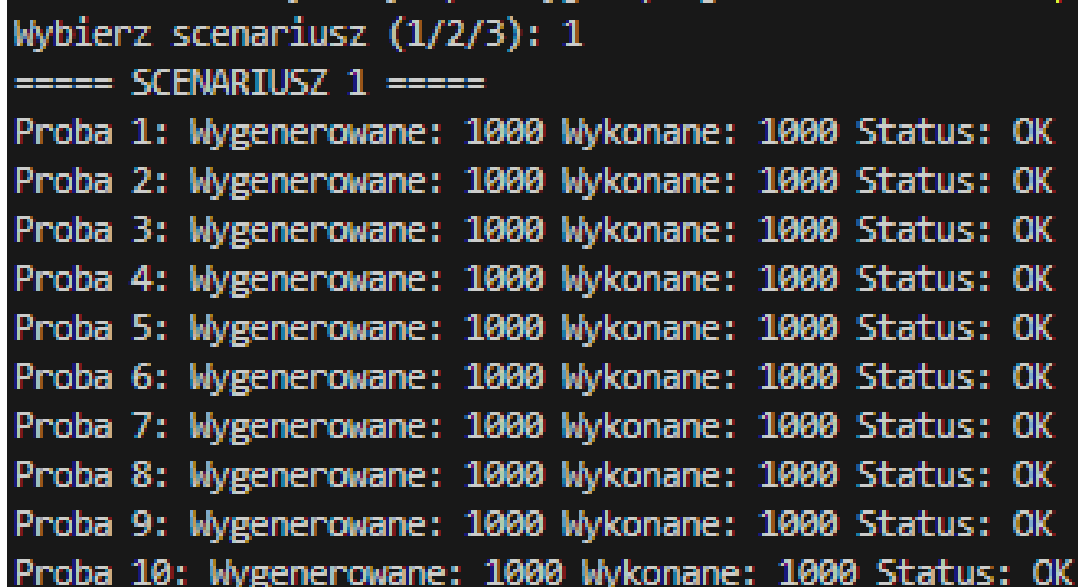
Spójność przetwarzania zapewnia relacja Wygenerowane = Wykonane raportowana po zakończeniu programu. Łagodne zamykanie polega na zamknięciu lokalnych kolejek (`close()`) i dołączeniu wątków (`join()`), co gwarantuje, że wszystkie przyjęte zadania zostaną obsłużone zanim program zakończy pracę. Zmodernizowano klasę `Logger`, wprowadzając **mechanizm buforowania asynchronicznego**. Zamiast wykonywać kosztowną operację zapisu na dysk (I/O) przy każdym wywołaniu `log()`, komunikaty są tymczasowo składowane w kontenerze `std::vector` w pamięci operacyjnej (RAM). Fizyczny zrzut danych do pliku tekstowego następuje jednorazowo, po zakończeniu pracy wszystkich wątków (metoda `dumpToFile`). Zaleta: Eliminuje to wąskie gardło związane z dostępem do dysku twardego, co jest kluczowe przy testach z udziałem 100 wątków. Dzięki temu wątki rywalizują o zasoby logiczne programu, a nie o dostęp do kontrolera dysku. Wada: W przypadku awarii zasilania lub nagłego przerwania programu przed procedurą zamknięcia, logi zgromadzone w buforze zostaną utracone, ponieważ nie zostały jeszcze utrwalone na nośniku.

5. Badania

Poniżej przedstawiono wyniki uzyskane dla trzech scenariuszy testowych. Pomiary mają postać wyjścia konsoli (logów).

5.1. Scenariusz 1: test spójności

Wynik pomiaru stanowią raporty końcowe dla kolejnych prób uruchomienia. Dla każdej próby rejestrowane są: liczba wygenerowanych zadań, liczba wykonanych zadań oraz status zgodności.



```
Wybierz scenariusz (1/2/3): 1
===== SCENARIUSZ 1 =====
Proba 1: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 2: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 3: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 4: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 5: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 6: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 7: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 8: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 9: Wygenerowane: 1000 Wykonane: 1000 Status: OK
Proba 10: Wygenerowane: 1000 Wykonane: 1000 Status: OK
```

Rysunek 1: Scenariusz 1 - wynik w konsoli (kolejne próby i raporty końcowe).

5.2. Scenariusz 2: test obsługi priorytetów

Wynik pomiaru stanowią logi z wykonania, w tym informacja o dodaniu zadania HIGH do kolejki oraz dalsze logi przetwarzania zadań.

```
[DataService #11] Przetwarzam zadanie id=1099 priorytet=1 dane="data_task"
[DataService] odczyt idx 9 -> 2
[ReportService #47] Przetwarzam zadanie id=2099 priorytet=2 dane="report_task"
[ReportService] odczyt idx 9 -> 2
[TEST] Do kolejki DataService wchodzi zadanie HIGH: id=9999 priorytet=1000 dane=TEST_HIGH
[ReportService #0] Przetwarzam zadanie id=2100 priorytet=2 dane="report_task"
[ReportService] zapis idx 0 = 2
[DataService #19] Przetwarzam zadanie id=9999 priorytet=1000 dane="TEST_HIGH"
[DataService] odczyt idx 9 -> 2
[DataService #23] Przetwarzam zadanie id=1100 priorytet=10 dane="data_task"
[DataService] odczyt idx 0 -> 2
[DataService #20] Przetwarzam zadanie id=1101 priorytet=1 dane="data_task"
[DataService] zapis idx 1 = 1
```

Rysunek 2: Scenariusz 2 - wynik w konsoli (log dodania zadania HIGH i przebieg przetwarzania).

5.3. Scenariusz 3: test procedury zamykania

Wynik pomiaru stanowią logi z wykonania, w tym komunikat informujący o zakończeniu przyjmowania nowych zadań wypisywany po dodaniu ostatnich zadań do kolejek. Dalsze logi pokazują przetwarzanie zadań znajdujących się już w kolejce oraz końcowy raport zgodności liczników.

```
[DataService #13] Przetwarzam zadanie id=1498 priorytet=1 dane="data_task"
[DataService] zapis idx 8 = 1
[ReportService #49] Przetwarzam zadanie id=2498 priorytet=2 dane="report_task"
[ReportService] zapis idx 8 = 2
[TEST] Koniec dodawania nowych zadan
[DataService #8] Przetwarzam zadanie id=1499 priorytet=1 dane="data_task"
[DataService] zapis idx 9 = 1
[ReportService #2] Przetwarzam zadanie id=2499 priorytet=2 dane="report_task"
[ReportService] zapis idx 9 = 2
```

Rysunek 3: Scenariusz 3 - wynik w konsoli (komunikat oraz dokończenie pracy przez wątki).

6. Wnioski

Na podstawie przeprowadzonych testów i obserwacji logów można sformułować następujące wnioski:

- Mechanizmy synchronizacji zastosowane w programie zapewniają spójność przetwarzania: we wszystkich zaprezentowanych uruchomieniach końcowe liczniki spełniają zależność Wygenerowane = Wykonane.
- Zastosowanie lokalnych kolejek priorytetowych w usługach pozwala sterować kolejnością pobierania zadań w obrębie danej usługi (zadania o wyższym priorytecie mają pierwszeństwo w kolejce).
- W scenariuszu 2 widoczny jest wpływ współbieżności i braku wywłaszczania: czasami pomiędzy logiem informującym o dodaniu zadania HIGH do kolejki a logiem jego przetworzenia pojawiają się logi innych, losowo wykonywanych zadań. Wynika to z tego, że priorytet porządkuje kolejność pobrania zadań z kolejki, ale nie przerywa zadania już wykonywanego oraz nie eliminuje przeplatania logów pochodzących z wielu wątków.
- Scenariusz 3 pokazuje rozdzielenie fazy generowania pracy od fazy przetwarzania: po informacji o zakończeniu przyjmowania nowych zadań wciąż pojawiają się logi przetwarzania, aż do całkowitego opróżnienia kolejki przez wątki robocze.
- Zamykanie programu jest realizowane w sposób łagodny: zatrzymanie polega na zamknięciu kolejki i dołączeniu wątków, co zapobiega utracie przyjętych zadań.

7. Podsumowanie

W ramach projektu zaimplementowano wielowątkowy serwer usług w języku C++, w którym każda usługa posiada własną kolejkę priorytetową zadań i pulę wątków roboczych. Zasoby współdzielone (logger oraz pula danych) zostały zabezpieczone odpowiednimi prymitywami synchronizacji, w tym modelem czytelnicy-pisarz dla danych globalnych. Przygotowane scenariusze testowe oraz zrzuty ekranu z uruchomień potwierdzają poprawność działania systemu, spójność liczników zadań oraz obserwowalny wpływ priorytetów i współbieżności na kolejność oraz przeplatanie logów.

8. Literatura

- Wikipedia: Wielowątkowość
- Wikipedia: Condition variables
- Dariusz Caban, Systemy operacyjne - prezentacje z wykładu (materiały dydaktyczne).

9. Kod

9.1. main.cpp

```
1  #include <iostream>
2  #include <filesystem>
3  #include <iomanip>
4  #include <sstream>
5  #include <thread>
6  #include <chrono>
7
8  #include "Task.hpp"
9  #include "SharedResources.hpp"
10 #include "Service.hpp"
11
12 int main() {
13     const bool testMode = false;           // flaga do uruchomienia trybu
14     const bool terminalLogMode = false;     // flaga do sterowania wypisywaniem
15                                             // logow w terminalu
16
17     auto prepare_log_file_path = []() -> std::filesystem::path {
18         namespace fs = std::filesystem;
19
20         fs::path logsDir = "logs";
21         try {
22             if (!fs::exists(logsDir)) {
23                 fs::create_directories(logsDir);
24             }
25         } catch (...) {
26             std::cerr << "Wystapil blad.\n";
27             return {};
28         }
29
30         auto now = std::chrono::system_clock::now();
31         std::time_t t = std::chrono::system_clock::to_time_t(now);
32         std::tm localTm{};
33         localtime_s(&localTm, &t);
34
35         std::ostringstream name;
36         name << "logs-"
37             << std::setw(2) << std::setfill('0') << localTm.tm_mday << "-"
38             << std::setw(2) << std::setfill('0') << (localTm.tm_mon + 1) <<
39             << "-"
40             << (localTm.tm_year + 1900) << "-"
41             << std::setw(2) << std::setfill('0') << localTm.tm_hour << "-"
42             << std::setw(2) << std::setfill('0') << localTm.tm_min
43             << ".txt";
44
45         return logsDir / name.str();
46     };
47
48     const std::filesystem::path logFilePath = prepare_log_file_path();
```

```

47
48     auto run_default_program = [&](bool quietLogs, bool injectHighPriority,
49     bool logEndOfSubmitting, std::chrono::milliseconds stopAfter, int&
50     producedOut, int& doneOut) {
51         const int taskPairs = 500;           // liczba par zadan
52
53         SharedResources shared;              // wspólne zasoby
54         shared.logger.setEnabled(!quietLogs); // wyłączenie logów w 1
55         scenariuszu
56         shared.logger.setConsoleEnabled(terminalLogMode && !quietLogs);
57
58         Service dataService("DataService", shared, 50);           // usługa
59         przetwarzająca dane
60         Service reportService("ReportService", shared, 50);       // usługa
61         generująca raporty
62
63         dataService.start();
64         reportService.start();
65
66         for (int i = 0; i < taskPairs; ++i) {
67             Task t1{ i + 1000, (i % 5 == 0) ? 10 : 1, "data_task" };
68             Task t2{ i + 2000, (i % 7 == 0) ? 7 : 2, "report_task" };
69             dataService.submit(t1);
70             reportService.submit(t2);
71
72             // scenariusz 2: wstrzyknięcie zadania o wysokim priorytecie
73             jako i + 1 zadanie
74             if (injectHighPriority && i == 100) {
75                 shared.logger.log("[TEST] Do kolejki DataService wchodzi
76                 zadanie HIGH: id=9999 priorytet=1000 dane=TEST_HIGH");
77                 Task high{ 9999, 1000, "TEST_HIGH" };
78                 dataService.submit(high);
79             }
80
81             // scenariusz 3: informacja o końcu dodawania zadań
82             if (logEndOfSubmitting && (i + 1) == taskPairs) {
83                 shared.logger.log("[TEST] Koniec dodawania nowych zadan");
84             }
85
86             std::this_thread::sleep_for(std::chrono::milliseconds(5));
87         }
88
89         if (logEndOfSubmitting && taskPairs <= 0) {
90             shared.logger.log("[TEST] Koniec dodawania nowych zadan");
91         }
92
93         std::this_thread::sleep_for(stopAfter);
94
95         dataService.stop();
96         reportService.stop();
97
98         // podsumowanie wyników
99         producedOut = dataService.produced_count() + reportService.
100         produced_count();
101         doneOut = dataService.done_count() + reportService.done_count();
102
103         // Zrzut bufora logow do pliku

```

```

97         if (!logFilePath.empty()) {
98             shared.logger.dumpToFile(logFilePath, true);
99         }
100     };
101
102     auto print_test_header = [](int scenario) {
103         std::cout << "==== SCENARIUSZ " << scenario << " =====\n";
104     };
105
106     auto print_summary = [](int produced, int done) {
107         std::cout << "Wygenerowane: " << produced << " Wykonane: " << done
108         << " Status: " << (produced == done ? "OK" : "MISMATCH") << "\n";
109     };
110
111     if (testMode) {
112         std::cout << "Wybierz scenariusz (1/2/3): ";
113
114         int scenario = 0;
115         if (!(std::cin >> scenario)) return 1;
116
117         int runs = 1;
118         bool quietLogs = false;
119         bool injectHighPriority = false;
120         bool logEndOfSubmitting = false;
121         std::chrono::milliseconds stopAfter = std::chrono::seconds(1);
122
123         switch (scenario) {
124             case 1:
125                 runs = 10;
126                 quietLogs = true;
127                 stopAfter = std::chrono::seconds(1);
128                 break;
129             case 2:
130                 injectHighPriority = true;
131                 stopAfter = std::chrono::seconds(1);
132                 break;
133             case 3:
134                 logEndOfSubmitting = true;
135                 stopAfter = std::chrono::milliseconds(150);
136                 break;
137             default:
138                 std::cout << "Niepoprawny numer scenariusza.\n";
139                 return 1;
140         }
141
142         print_test_header(scenario);
143
144         for (int i = 1; i <= runs; ++i) {
145             int produced = 0;
146             int done = 0;
147             run_default_program(quietLogs, injectHighPriority,
148             logEndOfSubmitting, stopAfter, produced, done);
149             if (runs > 1) {
150                 std::cout << "Proba " << i << ": ";
151             }
152             print_summary(produced, done);
153         }

```

```

153         return 0;
154     }
155
156     int produced = 0;
157     int done = 0;
158     run_default_program(false, false, false, std::chrono::seconds(1),
159         produced, done);
160     print_summary(produced, done);
161     return 0;
162 }

```

9.2. Task.hpp

```

1  #pragma once
2  #include <string>
3
4  struct Task {
5      int id;
6      int priority;
7      std::string data;
8  };
9
10 // komparator dla kolejki priorytetowej
11 struct TaskCompare {
12     bool operator()(const Task& a, const Task& b) const {
13         // wyższy priorytet ma większą wartość
14         return a.priority < b.priority;
15     }
16 };

```

9.3. SafePriorityQueue.hpp

```

1  #pragma once
2  #include <queue>
3  #include <vector>
4  #include <mutex>
5  #include <condition_variable>
6
7  template <typename T, typename Compare = std::less<T>>
8  class SafePriorityQueue {
9  public:
10     SafePriorityQueue() = default;
11
12     // dodanie elementu
13     void push(const T& item) {
14         {
15             std::lock_guard<std::mutex> lock(mtx);
16             if (closed) return; // jeśli kolejka jest zamknięta to nie
17             dodajemy
18             queue.push(item);
19             cvNotEmpty.notify_one();
20         }
21
22         // próba pobrania elementu

```



```

23     bool try_pop(T& out) {
24         std::lock_guard<std::mutex> lock(mtx);
25         if (queue.empty()) return false;
26         out = queue.top();
27         queue.pop();
28         return true;
29     }
30
31     // pobranie elementu z oczekiwaniem
32     bool wait_and_pop(T& out) {
33         std::unique_lock<std::mutex> lock(mtx);
34         cvNotEmpty.wait(lock, [this] { return closed || !queue.empty(); });
35         // czekaj aż kolejka nie będzie pusta lub zostanie zamknięta
36         if (queue.empty()) return false;
37         out = queue.top();
38         queue.pop();
39         return true;
40     }
41
42     // zamknięcie kolejki
43     void close() {
44         {
45             std::lock_guard<std::mutex> lock(mtx);
46             closed = true;
47         }
48         cvNotEmpty.notify_all();
49     }
50
51     // sprawdzenie czy kolejka jest pusta
52     bool empty() const {
53         std::lock_guard<std::mutex> lock(mtx);
54         return queue.empty();
55     }
56
57     // pobranie rozmiaru kolejki
58     size_t size() const {
59         std::lock_guard<std::mutex> lock(mtx);
60         return queue.size();
61     }
62
63     // sprawdzenie czy kolejka jest zamknięta
64     bool isClosed() const {
65         std::lock_guard<std::mutex> lock(mtx);
66         return closed;
67     }
68 private:
69     std::priority_queue<T, std::vector<T>, Compare> queue;      // wewnętrzna
70     kolejka priorytetowa
71     mutable std::mutex mtx;                                     // mutex do synchronizacji dostę
72     pu
73     std::condition_variable cvNotEmpty;                       // warunek do powiadamiania o
74     dostępności zasobów
75     bool closed = false;                                       // flaga zamknięcia kolejki
76 };

```

9.4. SharedResources.hpp

```
1  #pragma once
2  #include <filesystem>
3  #include <fstream>
4  #include <mutex>
5  #include <shared_mutex>
6  #include <string>
7  #include <vector>
8  #include <iostream>
9
10 class SharedResources {
11 public:
12     class Logger {
13     public:
14         // włączenie/wyłączenie logów (wspólne dla terminala i pliku)
15         void setEnabled(bool value) {
16             std::lock_guard<std::mutex> lock(logMtx);
17             enabled = value;
18         }
19
20         // sterowanie wypisywaniem logów w terminalu
21         void setConsoleEnabled(bool value) {
22             std::lock_guard<std::mutex> lock(logMtx);
23             consoleEnabled = value;
24         }
25
26         // wypisanie logu (buforowanie w pamięci + opcjonalnie terminal)
27         void log(const std::string& msg) {
28             std::lock_guard<std::mutex> lock(logMtx);
29             if (!enabled) return;
30             logBuffer.push_back(msg);
31             if (consoleEnabled) {
32                 std::cout << msg << std::endl;
33             }
34         }
35
36         // zapis bufora do pliku
37         void dumpToFile(const std::filesystem::path& filePath, bool append =
true) {
38             std::lock_guard<std::mutex> lock(logMtx);
39             if (!enabled) return;
40             if (filePath.empty()) return;
41             if (logBuffer.empty()) return;
42
43             std::ofstream file;
44             file.open(filePath, append ? (std::ios::out | std::ios::app) :
std::ios::out);
45             if (!file.is_open()) {
46                 if (consoleEnabled) {
47                     std::cerr << "Nie udało się otworzyć pliku logów do
zapisu: " << filePath.string() << std::endl;
48                 }
49                 return;
50             }
51
52             for (const auto& line : logBuffer) {
53                 file << line << "\n";
```

```

54         }
55         file.close();
56         logBuffer.clear();
57     }
58     private:
59         std::mutex logMtx;           // mutex do synchronizacji wypisywania logów
60         bool enabled{true};         // flaga włączenia/wyłączenia logów
61         bool consoleEnabled{true};
62         std::vector<std::string> logBuffer;
63     };
64
65     class DataPool {
66     public:
67         int read(size_t idx) const {
68             std::shared_lock<std::shared_mutex> lock(sharedMtx);
69             if (idx < dataVector.size()) return dataVector[idx];
70             return -1;
71         }
72         void write(size_t idx, int value) {
73             std::unique_lock<std::shared_mutex> lock(sharedMtx);
74             if (idx >= dataVector.size()) dataVector.resize(idx + 1, 0);
75             dataVector[idx] = value;
76         }
77         size_t size() const {
78             std::shared_lock<std::shared_mutex> lock(sharedMtx);
79             return dataVector.size();
80         }
81     private:
82         mutable std::shared_mutex sharedMtx;    // mutex do synchronizacji
83         dostępu do danych                       // wektor przechowujący dane
84         std::vector<int> dataVector;
85     };
86     Logger logger;
87     DataPool data;
88 };

```

9.5. Service.hpp

```

1  #pragma once
2  #include <vector>
3  #include <thread>
4  #include <mutex>
5  #include <string>
6  #include <random>
7  #include <chrono>
8
9  #include "Task.hpp"
10 #include "SafePriorityQueue.hpp"
11 #include "SharedResources.hpp"
12
13 // klasa reprezentująca usługę przetwarzającą zadania
14 class Service {
15 public:
16     Service(std::string name, SharedResources& shared, int workerCount);
17     ~Service();

```

```

18
19     void start();
20     void stop();
21
22     void submit(const Task& task);           // dodanie zadania do kolejki
23
24     int produced_count() const;             // liczba stworzonych zadań
25     int done_count() const;                // liczba wykonanych zadań
26
27     // oczekiwanie na wykonanie określonej liczby zadań (do testów)
28     bool wait_until_done(int expectedDone, std::chrono::milliseconds timeout
29 ) const;
30
31     const std::string& name() const { return serviceName; }
32
33 private:
34     void worker_loop(int workerId);         // funkcja wykonywana przez wątki
35     robocze
36
37     std::string serviceName;
38     SharedResources& sharedResources;
39     SafePriorityQueue<Task, TaskCompare> taskQueue;
40     std::vector<std::thread> workers;
41     mutable std::mutex mtx;
42     int producedCount{0};
43     int doneCount{0};
44     bool running{false};
45 };

```

9.6. Service.cpp

```

1  #include "Service.hpp"
2
3  Service::Service(std::string name, SharedResources& shared, int workerCount)
4      : serviceName(std::move(name)), sharedResources(shared) {
5      workers.reserve(workerCount);
6      for (int i = 0; i < workerCount; ++i) {
7          workers.emplace_back();
8      }
9  }
10
11  Service::~Service() {
12      stop();
13  }
14
15  void Service::start() {
16      {
17          std::lock_guard<std::mutex> lock(mtx);
18          running = true;
19      }
20      for (size_t i = 0; i < workers.size(); ++i) {
21          workers[i] = std::thread(&Service::worker_loop, this, static_cast<
22 int>(i));
23      }
24  }

```

```

25 void Service::stop() {
26     {
27         std::lock_guard<std::mutex> lock(mtx);
28         if (!running) {
29             // zatrzymane wcześniej
30         } else {
31             running = false;
32         }
33     }
34     taskQueue.close();
35     for (auto& t : workers) {
36         if (t.joinable()) t.join();
37     }
38 }
39
40 // dodanie nowego zadania do kolejki
41 void Service::submit(const Task& task) {
42     std::lock_guard<std::mutex> lock(mtx);
43     if (running) {
44         ++producedCount;
45         taskQueue.push(task);
46     }
47 }
48
49 int Service::produced_count() const {
50     std::lock_guard<std::mutex> lock(mtx);
51     return producedCount;
52 }
53 int Service::done_count() const {
54     std::lock_guard<std::mutex> lock(mtx);
55     return doneCount;
56 }
57
58 // oczekiwanie na wykonanie określonej liczby zadań
59 bool Service::wait_until_done(int expectedDone, std::chrono::milliseconds
    timeout) const {
60     auto deadline = std::chrono::steady_clock::now() + timeout;
61     while (std::chrono::steady_clock::now() < deadline) {
62         if (done_count() >= expectedDone) return true;
63         std::this_thread::sleep_for(std::chrono::milliseconds(10));
64     }
65     return done_count() >= expectedDone;
66 }
67
68 // funkcja wykonywana przez wątki robocze
69 void Service::worker_loop(int workerId) {
70     std::mt19937 rng(std::random_device{}()); // generator liczb
    losowych
71     std::uniform_int_distribution<int> dist(0, 1); // 0 - read, 1 -
    write
72     Task task;
73     // przetwarzanie zadań
74     while (taskQueue.wait_and_pop(task)) {
75         sharedResources.logger.log "[" + serviceName + " #" + std::to_string
            (workerId) + "] Przetwarzam zadanie id=" + std::to_string(task.id) + "
            priorytet=" + std::to_string(task.priority) + " dane=\"" + task.data + "
            \""");
76         if (dist(rng) == 0) {

```

```

77         // read
78         size_t idx = static_cast<size_t>(task.id % 10);
79         int val = sharedResources.data.read(idx);
80         sharedResources.logger.log "[" + serviceName + "] odczyt idx " +
std::to_string(idx) + " -> " + std::to_string(val));
81     } else {
82         // write
83         size_t idx = static_cast<size_t>(task.id % 10);
84         sharedResources.data.write(idx, task.priority);
85         sharedResources.logger.log "[" + serviceName + "] zapis idx " +
std::to_string(idx) + " = " + std::to_string(task.priority));
86     }
87     std::this_thread::sleep_for(std::chrono::milliseconds(20));    //
symulacja czasu przetwarzania
88     {
89         std::lock_guard<std::mutex> lock(mtx);
90         ++doneCount;
91     }
92 }
93 sharedResources.logger.log "[" + serviceName + " #" + std::to_string(
workerId) + "] koncze prace");    // wypis przy wychodzeniu
94 }

```